Security Audit

CROPSETU — multi-seller agri-marketplace

Scope	backend/ · frontend/ · seller-app/ · admin/ · shared/
Stack	Node 20 · Express 4 (ESM) · Prisma 5 · PostgreSQL · Redis · Expo / React Native · React + Vite
Method	White-box source review with full repository access
Branch	feat/rent-location-filter
Date	19 August 2026
Findings	2 Critical · 3 High · 5 Medium · 3 Low

> **Authorised review.** This document describes exploitable defects in the owner's own application, written for remediation. It contains working attack descriptions — treat it as confidential and do not distribute outside the engineering team. The two Critical findings are live financial exposure and should be fixed ahead of all other work.

Summary of findings

ID	SEVERITY	FINDING	LOCATION
C-1	CRITICAL	Payment signature verification is bypassable in mock mode	`payment.service.js:31`
C-2	CRITICAL	Order confirmation is replayable — one payment, many orders	`agristore.routes.js`
H-1	HIGH	Stock can be driven negative — no floor in the raw UPDATE	`stockBatch.js:37`
H-2	HIGH	Cart add is a check-then-act race	`agristore.routes.js:210`
H-3	HIGH	Reviews require no purchase — search ranking is manipulable	`agristore.routes.js:995`
M-1	MEDIUM	Upload accepts arbitrary bytes as an image	`upload.routes.js:22`
M-2	MEDIUM	Missing UUID guard yields a 500 error oracle	`agristore.routes.js:41`
M-3	MEDIUM	Order status rollup races and silently destroys inventory	`agristore.routes.js:929`
M-4	MEDIUM	Duplicate seller payouts are possible	`schema.prisma` · Payout
M-5	MEDIUM	Soft-deleted products remain publicly fetchable	`agristore.routes.js:173`
L-1	LOW	Checkout serialization conflicts surface as 500s	`agristore.routes.js:376`
L-2	LOW	Referential integrity unenforced on the money rails	`schema.prisma:2168`
L-3	LOW	`minOrderQty` is stored but never enforced	`agristore.routes.js`

Findings

C-1 · Payment signature verification is bypassable CRITICAL

`backend/src/services/payment.service.js:31` — CWE-287 Improper Authentication · CWE-1188 Insecure Default

MECHANISM

```
const isMock = !ENV.RAZORPAY_KEY_ID || !ENV.RAZORPAY_KEY_SECRET;
```

In mock mode `verifyPaymentSignature()` returns `true` unconditionally, and `fetchPaymentOrder()` returns `{ id, mock: true }` with **no amount**. `POST /agristore/orders/confirm` gates both of its anti-tamper bindings behind `if (!paymentOrder.mock)`, so in mock mode the receipt binding (which blocks replaying another user's payment id) and the paid-amount binding (which blocks initiate-cheap-then-enlarge-cart) are **both skipped**.

ATTACK

> Precondition: production boots with either Razorpay variable unset or empty. No special role needed — an ordinary logged-in buyer.
>
> 1. Fill a cart with any goods.
> 2. `POST /api/v1/agristore/orders/confirm` with arbitrary strings for `razorpayOrderId`, `razorpayPaymentId` and `razorpaySignature`.
> 3. Signature check passes unconditionally; both amount bindings are skipped.
> 4. Order is written with `paymentStatus: 'paid'`. Stock decrements, the seller ships.
>
> **Impact:** unlimited free checkout across every seller on the platform. Direct, unbounded financial loss with no rate limit on value.

FIX

1. Add Razorpay key presence to the production config validator in `src/config/env.js`. It already throws `FATAL: production config invalid` — the process must refuse to boot rather than silently degrading to "signature always valid".
2. Add an independent guard in `payment.service.js`: when `NODE_ENV === 'production'` and `isMock`, throw on every call.
3. Regression tests: mock mode under production config fails boot; a bad signature returns false in every environment.

C-2 · Order confirmation is replayable CRITICAL

`backend/src/routes/agristore.routes.js — POST /orders/confirm — CWE-294 Capture-Replay · CWE-799 Improper Control of Interaction Frequency`

MECHANISM

The Razorpay payment id is persisted only inside the free-text `notes` field (`notes: \`razorpay:${razorpayPaymentId}\``). There is no unique constraint on it and no dedupe check anywhere on the route. The HMAC covers only `orderId|paymentId`, so a captured triple stays valid forever.

ATTACK

1. Complete one genuine payment. Capture the signed triple from your own client.
2. Re-fill the cart with anything.
3. Replay the identical triple to `/orders/confirm`.
4. HMAC validates. Receipt binding passes — same user, same `cart_{userId}` receipt. A second paid order is created.
5. Repeat indefinitely.

Impact: N orders for the price of one. Requires only one real payment to bootstrap, and scales without limit.

FIX

Add `Order.razorpayPaymentId String? @unique` — additive, safe for the `db push` deploy path — and write it as a real column. Treat the uniqueness violation as the idempotency signal and return the *existing* order rather than an error, so a legitimate client retry stays safe.

Test: the same signed triple submitted twice yields exactly one `Order` row; the second call returns the first order.

H-1 · Stock can be driven negative HIGH

`backend/src/utils/stockBatch.js:37-42 — CWE-20 Improper Input Validation`

MECHANISM

```
UPDATE products AS p SET stock = p.stock + v.delta
FROM (VALUES ...) AS v(id, delta) WHERE p.id = v.id
```

No `stock >= 0` guard. Correctness rests entirely on in-transaction validation plus Serializable isolation. Called from three places — COD checkout, payment confirm, and buyer cancel. One missed validation path, or any future caller of this helper, oversells silently with no error.

FIX

> Add `AND p.stock + v.delta >= 0` to the `WHERE`, and have the caller assert that the affected row count equals the number of deltas, aborting the transaction otherwise. Defence in depth — keep the existing in-transaction validation. Test: two concurrent checkouts of the last unit → one succeeds, stock lands at 0, never below.

H-2 · Cart add is a check-then-act race HIGH

`backend/src/routes/agristore.routes.js:210-232 — CWE-367 TOCTOU`

MECHANISM

`POST /cart` reads the product, computes `totalAfter = (existing?.quantity || 0) + quantity`, checks it against `product.stock`, then upserts with an `increment`. The read and the write are not in a transaction.

ATTACK

> Fire N concurrent `POST /cart` requests. All N pass the stock check against the same stale read; every increment applies. The cart holds more units than exist. Checkout's Serializable re-validation eventually catches it — so this is not oversell — but the buyer hits a hard failure at the payment step instead of a clear message at add time, and any path trusting cart quantity in between is exposed.

FIX

> Wrap the read and upsert in a transaction, or express the stock check as a conditional write. Return a 400 carrying the real available quantity.

H-3 · Reviews require no purchase HIGH

`backend/src/routes/agristore.routes.js:995 — POST /products/:id/review — CWE-862 Missing Authorization`

MECHANISM

`authenticate` only. No `requireRole`, no purchase verification, and the `Review` model has no `orderId` — a purchase link is not even expressible in the schema. The preceding `product.findUnique` does not filter `isActive`.

ATTACK

> Any authenticated account rates any product 1–5. The route then recomputes `product.rating` and calls `bumpListingVersion(NS_PRODUCTS)` *precisely because rating drives the storefront sort order*. Ratings — and therefore search ranking — are directly manipulable by anyone with an account, including competitors, and including against soft-deleted products. Cheap to automate: one account per review, no purchase, no cost.

FIX

> Add `Review.orderItemId String?` (additive), require a `DELIVERED` `OrderItem` for that user and product before accepting the review, and filter `isActive`.

M-1 · Upload accepts arbitrary bytes as an image MEDIUM

`backend/src/routes/upload.routes.js:22-24 — CWE-434 Unrestricted Upload of Dangerous File Type`

MECHANISM

```
if (base64.startsWith('data:') && !base64.startsWith('data:image/')) {
  return sendError(res, 'Only image files are allowed', 400);
}
```

The check reads the **prefix string only**, never the decoded magic bytes — and is skipped entirely when the payload has no `data:` prefix, which the code explicitly anticipates, since it strips the prefix conditionally on the next line.

ATTACK

> Send `<any bytes>`, or raw base64 with no prefix at all. Content type is entirely attacker-declared. Severity is held at Medium because Cloudinary re-encoding likely neutralises the payload in production — but the `!ENV.CLOUDINARY_CLOUD_NAME` branch returns a placeholder *before* any upload, so that mitigation is configuration-dependent.

FIX

> Sniff the decoded buffer's magic bytes; allow only JPEG, PNG and WebP. Reject SVG explicitly — a stored-XSS vector if the asset is ever served inline.

M-2 · Missing UUID guard yields a 500 error oracle MEDIUM

`backend/src/routes/agristore.routes.js:41-42 — CWE-209 Information Exposure Through an Error Message`

`router.param` registers `uuidParamGuard` for `id` and `productId` only. `PUT /seller/orders/:orderId/status` therefore accepts any string; a non-UUID reaches `prisma.orderItem.updateMany` against a uuid column and, with no try/catch on the route, surfaces as a 500 through the global handler.

FIX

> Add `router.param('orderId', uuidParamGuard)`, then sweep **every** route file for `router.param` coverage of every uuid parameter. This is a class of defect, not a single instance.

M-3 · Status rollup races and destroys inventory MEDIUM

`backend/src/routes/agristore.routes.js:929-957 — CWE-362 Race Condition`

Three defects in one handler:

- The read-derive-write of `Order.status` runs **outside a transaction** — two sellers updating the same multi-seller order concurrently persist a stale rollup.
- A partially-cancelled order falls through to `PENDING`, so the buyer sees "pending" for an order one seller already cancelled.
- A seller setting `CANCELLED` **never restocks.** Unlike the buyer-cancel path, `applyStockDeltas` is never called — inventory is silently destroyed on every seller-side cancellation.

FIX

> Wrap the rollup in a transaction, represent partial cancellation explicitly instead of falling through, and call `applyStockDeltas` with positive deltas on seller cancellation.

M-4 · Duplicate seller payouts are possible MEDIUM

`backend/prisma/schema.prisma — model Payout — CWE-837 Improper Enforcement of a Single Unique Action`

No `@@unique`. Nothing prevents two payout rows for the same `sellerId` over the same `periodFrom/periodTo` — a double-payout is one retried admin action or one duplicated cron run away.

FIX

> Add `@@unique([sellerId, periodFrom, periodTo])` (additive). Check for existing duplicates first — the constraint fails to apply if any exist.

M-5 · Soft-deleted products remain publicly fetchable MEDIUM

`backend/src/routes/agristore.routes.js:173 — GET /products/:id — CWE-200 Exposure of Sensitive Information`

No auth middleware and no `isActive` filter. `DELETE /seller/products/:id` is a soft delete (`isActive: false`), so every "deleted" product stays retrievable by id, including seller-supplied description and images.

FIX

> Filter `isActive: true` for non-owner requests; decide deliberately whether owner and admin may still fetch.

L-1 · Checkout serialization conflicts surface as 500s LOW

`backend/src/routes/agristore.routes.js:376, 653`

Both checkout paths use `prisma.$transaction(fn, { isolationLevel: 'Serializable' })` with no retry wrapper. Two buyers racing for the last unit is a normal event, but a 40001 abort returns a 500 "Checkout failed. Please try again." Availability defect, not a breach.

FIX

> Bounded retry (2–3 attempts with jitter) around the transaction; return 409, not 500, on genuine conflict.

L-2 · Referential integrity unenforced on the money rails LOW

`backend/prisma/schema.prisma:2168-2172 — SellerLedgerEntry · Payout · Dispute`

All three use bare `String` scalars with no foreign keys — deliberately, to keep `db push` additive. Orphaned ledger entries and payouts to deleted users are possible.

FIX

> **Do not add FKs** — that breaks the deploy path. Add a periodic integrity check reporting orphans, and validate the referenced user exists at write time.

L-3 · `minOrderQty` is stored but never enforced LOW

`backend/src/routes/agristore.routes.js:712, 751, 818, 851`

Accepted and persisted on product create and update, read by no cart or checkout path. Quantity is bounded only by the validator's `1..100` and by stock. Sellers believe they have a minimum order quantity that does not exist.

FIX

> Enforce in cart add, cart update and checkout — or remove the field.

Requires verification

Not confirmed as exploitable. Verify before changing.

- **CSRF pre-auth exemption** — `backend/src/middleware/csrf.js`. Matches with `path.endsWith(suffix)` over `PRE_AUTH_PATHS`. `csrfProtection` is global middleware mounted at `src/app.js:326`, *before* routing, so `req.path` is the full request path. Confirm no other route can end with `/auth/send-otp` or `/auth/verify-otp`, and consider exact-matching against the mounted API prefix instead of `endsWith`. The exemption itself is sound: those endpoints establish a session rather than act on one, and `/auth/refresh` correctly stays protected.

Verified sound — do not "harden" these

Each was examined and found correct. Changing them risks introducing a regression.

- **No SQL injection in `keysetPage`** — `src/utils/keyset.js:72` uses `$queryRawUnsafe`, but identifiers are regex-guarded by `IDENT` and all values are parameterised. Not injectable.
- **No secrets in git history** — no `.env`, key or certificate file has ever been committed.
- **Production config is gated** — `src/config/env.js` validates production configuration and throws `FATAL` on invalid setups. JWT secret length is enforced at ≥ 32 characters.
- **Field encryption is sound** — AES-256-GCM with a key-id registry and rotation support.
- **Authentication is strong** — OTP has proof-of-work gating plus IP and phone rate limiters on both send and verify. Refresh tokens rotate with reuse detection, token-family revocation, and a security-incident record on replay.
- **Transport and headers** — CORS uses an allowlist that refuses to reflect arbitrary origins in production; `helmet` and `trust proxy` are configured.

Remediation order

#	ACTION	FINDINGS
1	Ship alone, ahead of all other work — live financial exposure	C-1, C-2
2	Integrity and abuse controls	H-1, H-2, H-3
3	`router.param` sweep across all route files	M-2
4	Remaining medium findings	M-1, M-3, M-4, M-5
5	Availability and hygiene	L-1, L-2, L-3

Constraint on every fix: production applies schema with `prisma db push` (`package.json` → `start:prod`), not `migrate deploy`. Only additive schema changes are safe; a non-additive change hits the data-loss guard and applies *nothing*. This has already caused an outage — `users.adminScopes` went missing in production and every login returned 500 with P2022. Non-additive work belongs in `backend/prisma/manual/*.sql` as hand-written drop-free SQL. Never pass `--accept-data-loss`.